Ejercicio Demostrativo

Matriz dispersa — Unidad 4

Auxiliar: Sergio Sebastian Sandoval Ruiz

Tiempo estimado: 60 min

5 minutos para Solidaridad y 55 minutos de desarrollo académico

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas
Segundo Semestre 2026
Laboratorio de Estructuras de Datos C

1. Marco Formativo

1.1 Valores

Solidaridad	Apoyar a los demás, especialmente en momentos difíciles. Durante la práctica, explicar con paciencia las coordenadas y los índices, revisar el procedimiento del compañero y ayudarlo a encontrar errores sin reemplazar su trabajo.

1.2 Competencias

Tipo de competencia	Redacción
Competencia general	Implementa estructuras de datos lineales y no lineales mediante programación orientada a objetos para desarrollar soluciones eficientes.
Competencia específica	Representa una matriz dispersa mediante coordenadas en arreglos, implementa consulta, asignación y eliminación, y verifica transposición, suma y producto matriz-vector. Interpreta representaciones CSR y la organización ortogonal.

2. Palabras Clave

Matriz dispersa: Matriz con suficientes ceros para justificar una representación que omita la mayoría de ellos.

Tripleta: Registro de fila, columna y valor de una entrada.

Cero implícito: Valor de una coordenada válida que no está almacenada.

nnz: Cantidad de entradas no nulas; en este ejercicio no se guardan ceros explícitos.

COO: Representación mediante arreglos de coordenadas y valores.

CSR y CSC: Formatos comprimidos organizados por filas y por columnas.

Transposición: Intercambio de filas y columnas sin cambiar los valores.

Estructura ortogonal: Nodos conectados tanto por fila como por columna.

3. Resumen

La demostración de la unidad 4 construye una matriz dispersa en Java y representa cada entrada mediante fila, columna y valor. El auxiliar modifica una matriz pequeña, comprueba los ceros implícitos y ejecuta operaciones cuyos resultados pueden verificarse también de forma manual.

El auxiliar presenta una traza antes de ejecutar el programa y contrasta la salida con resultados calculados a mano. El estudiante debe explicar los cambios de estado y justificar las condiciones de cada operación.

4. Contenido

4.1 DESCRIPCIÓN DEL EJERCICIO

Crear una matriz de 4 filas y 5 columnas con las entradas (0,2,5), (1,0,8), (2,3,-2), (3,1,6) y (3,4,9). Insertar (1,4,7), actualizar (0,2) a 12 y asignar cero en (2,3). Después obtener la transpuesta, multiplicar por un vector y sumar una segunda matriz.

4.2 OBJETIVO DE LA DEMOSTRACIÓN

> Relacionar las coordenadas con su almacenamiento, conservar una única entrada por posición y eliminar las entradas que se vuelven cero. Verificar que la transposición intercambie dimensiones y que las operaciones respeten sus condiciones de compatibilidad.

4.3 RECURSOS NECESARIOS

- Java JDK 11 o superior, editor o IDE y terminal con javac y java.
- Dos archivos en el mismo directorio: `MatrizDispersa.java` y `Main.java`.
- Diapositivas de Matriz dispersa de la unidad 4.
- Pizarrón o papel para trazar índices, estados y resultados.

Distribución del tiempo

Actividad	Minutos
Solidaridad y participación	5
Conceptos y traza inicial	15
Implementación en Java	20
Pruebas en consola	10
Contraste y cálculo manual	5

Actividad	Minutos
Resultados y cierre	5

4.4 DESARROLLO PASO A PASO

La representación utiliza tres arreglos paralelos y un contador n. Los índices comienzan en cero. Una coordenada fuera de las dimensiones es un error; una coordenada válida ausente representa cero. El ejemplo no almacena ceros explícitos ni admite duplicados.

Paso 1 — Valor y participación (5 min)

Apoyar a los demás, especialmente en momentos difíciles. Durante la práctica, explicar con paciencia las coordenadas y los índices, revisar el procedimiento del compañero y ayudarlo a encontrar errores sin reemplazar su trabajo.

Solicitar una propuesta antes de mostrar la respuesta. Comparar los procedimientos con la misma entrada y cuidar que la revisión permita comprender el error y corregirlo.

Paso 2 — Conceptos y traza (15 min)

Construir primero la matriz base en el pizarrón. Identificar sus 20 posiciones posibles y sus cinco entradas no nulas. Calcular densidad 5/20 = 25 % y proporción de ceros 75 %. Explicar que el ahorro real también depende del espacio que ocupan los índices.

Operación	Entrada afectada	nnz después
Estado base	Cinco tripletas	5
Insertar	A`[1,4]` = 7	6
Actualizar	A`[0,2]` = 12	6
Asignar cero	A`[2,3]` = 0	5

Qué observar:

- No se pierde la relación entre los tres arreglos: fi[k], co[k] y va[k] describen una misma entrada.
- Actualizar conserva el número de entradas. Asignar cero a una posición existente elimina una tripleta.
- La eliminación copia la última tripleta en el lugar retirado; el orden físico puede cambiar.
- El arreglo admite hasta 20 entradas activas. No implementa crecimiento automático.

Paso 3 — Implementación (20 min)

Crear `MatrizDispersa.java`. La primera parte valida dimensiones e índices y define get y set. El método posicion recorre las tripletas activas; la estructura no usa una matriz densa para guardar sus datos.

java

```
public class MatrizDispersa {
    private final int filas, columnas;
    private final int[] fi = new int[20], co = new int[20];
    private final int[] va = new int[20];
    private int n;
    public MatrizDispersa(int filas, int columnas) {
        if (filas < 1 || columnas < 1)
            throw new IllegalArgumentException("Dimensiones
invalidas");
        this.filas = filas; this.columnas = columnas;
    }
    private void validar(int i, int j) {
        if (i < 0 || i >= filas || j < 0 || j >= columnas)
            throw new IndexOutOfBoundsException("Coordenada
invalida");
    }
    private int posicion(int i, int j) {
        for (int k = 0; k < n; k++)
            if (fi[k] == i && co[k] == j) return k;
        return -1;
    }
```

```
    public int get(int i, int j) {
        validar(i,j);
        int k = posicion(i,j);
        return k < 0 ? 0 : va[k];
    }
    public void set(int i, int j, int valor) {
        validar(i,j);
        int k = posicion(i,j);
        if (k >= 0) {
            if (valor != 0) va[k] = valor;
            else {
                n--;
                fi[k] = fi[n]; co[k] = co[n]; va[k] = va[n];
            }
        } else if (valor != 0) {
            if (n == va.length)
                throw new IllegalStateException("Capacidad agotada");
            fi[n] = i; co[n] = j; va[n] = valor; n++;
        }
    }
```

Continuación de la implementación:

La segunda parte implementa transposición, producto y suma. Los valores y resultados usan int: las pruebas pequeñas evitan desbordamiento aritmético. La matriz resultante conserva el límite didáctico de 20 entradas.

java

```
    public MatrizDispersa transpuesta() {
        MatrizDispersa t = new MatrizDispersa(columnas,filas);
        for (int k = 0; k < n; k++) t.set(co[k],fi[k],va[k]);
        return t;
    }
    public int[] multiplicar(int[] x) {
```

```
        if (x.length != columnas)
            throw new IllegalArgumentException("Vector incompatible");
        int[] y = new int[filas];
        for (int k = 0; k < n; k++) y[fi[k]] += va[k]*x[co[k]];
        return y;
    }
    public MatrizDispersa sumar(MatrizDispersa b) {
        if (filas != b.filas || columnas != b.columnas)
            throw new IllegalArgumentException("Matrices
incompatibles");
        MatrizDispersa c = new MatrizDispersa(filas,columnas);
        for (int k = 0; k < n; k++) c.set(fi[k],co[k],va[k]);
        for (int k = 0; k < b.n; k++) {
            int i = b.fi[k], j = b.co[k];
            c.set(i,j,c.get(i,j)+b.va[k]);
        }
        return c;
    }
    public int nnz() { return n; }
    public void mostrar() {
        for (int k = 0; k < n; k++)
            System.out.println("("+fi[k]+","+co[k]+","+va[k]+")");
    }
}
```

Copiar las dos partes consecutivamente en el archivo indicado. No agregar una segunda declaración de clase ni omitir las llaves de cierre. La prueba del siguiente paso utiliza esta misma implementación.

Paso 4 — Pruebas en consola (10 min)

Crear `Main.java` en el mismo directorio. Ejecutar las modificaciones y mostrar las tripletas finales. Comparar los resultados con los cálculos realizados antes de abrir el IDE. Las posiciones mostradas corresponden a índices base cero.

java

```java
public class Main {
    public static void main(String[] args) {
        MatrizDispersa a = new MatrizDispersa(4,5);
        a.set(0,2,5); a.set(1,0,8); a.set(2,3,-2);
        a.set(3,1,6); a.set(3,4,9);
        a.set(1,4,7); a.set(0,2,12); a.set(2,3,0);
        System.out.println("Tripletas finales:"); a.mostrar();
        System.out.println("No nulos: "+a.nnz());
        System.out.println("A[2,3]: "+a.get(2,3));
        int[] y = a.multiplicar(new int[]{1,2,3,4,5});
        System.out.print("Ax:");
        for (int v : y) System.out.print(" "+v);
        System.out.println();
        System.out.println("T[2,0]: "+a.transpuesta().get(2,0));
        MatrizDispersa b = new MatrizDispersa(4,5);
        b.set(0,2,-12); b.set(2,0,4);
        MatrizDispersa c = a.sumar(b);
        System.out.println("Suma[0,2]: "+c.get(0,2));
        System.out.println("Suma[2,0]: "+c.get(2,0));
        System.out.println("No nulos de suma: "+c.nnz());
    }
}
```

Compilar y ejecutar:

```
javac MatrizDispersa.java Main.java
java Main
```

Salida esperada:

```
Tripletas finales:
(0,2,12)
(1,0,8)
```

```
(1,4,7)
(3,1,6)
(3,4,9)
No nulos: 5
A[2,3]: 0
Ax: 36 43 0 57
T[2,0]: 12
Suma[0,2]: 0
Suma[2,0]: 4
No nulos de suma: 5
```

La suma usa una matriz B con B`[0,2]` = -12 y B`[2,0]` = 4. Se cancela una entrada y aparece otra, por lo que conserva cinco no nulos. El resultado del producto corresponde a la matriz modificada, no a la base inicial.

Paso 5 — Representaciones y cálculo manual (5 min)

Expresar la matriz modificada en CSR. Agrupar por fila da valores `[12,8,7,6,9]`, columnas `[2,0,4,1,4]` y row_ptr `[0,1,3,3,5]`. Los dos punteros consecutivos iguales muestran que la fila 2 está vacía. Este formato se interpreta manualmente; el programa conserva COO.

Para una estructura ortogonal, dibujar cada nodo como una sola entrada con enlaces a la siguiente de su fila y de su columna. Al borrar un nodo hay que retirarlo de ambas cadenas. La implementación entregada no contiene esos enlaces: permite contrastar dos representaciones del mismo dato.

Para calcular Ax con x = `[1,2,3,4,5]`, sumar solamente las contribuciones no nulas de cada fila. Se obtiene 12×3 = 36; 8×1 + 7×5 = 43; fila vacía = 0; y 6×2 + 9×5 = 57.

Comprobación	Resultado
Dimensiones de A	4 × 5
Dimensiones de la transpuesta	5 × 4
Entrada transpuesta	T`[2,0]` = A`[0,2]` = 12
Vector de entrada	5 componentes

Comprobación	Resultado
Vector resultado	4 componentes

Pedir una explicación del resultado, no únicamente su lectura. El cálculo manual debe utilizar exactamente los datos y el orden indicados. Si cambia una entrada, se deben actualizar también las expectativas de la prueba.

Observaciones Importantes

- No confundir índice inválido con cero implícito. La validación debe ocurrir antes de consultar o modificar.
- La suma exige dimensiones iguales; el producto requiere que el vector tenga tantas componentes como columnas.
- La implementación elimina ceros exactos. No utiliza una tolerancia de punto flotante.
- Una inserción por búsqueda lineal hace que convertir o sumar mediante llamadas repetidas a set tenga un costo mayor que un algoritmo especializado.

4.5 Tabla Comparativa

TABLA COMPARATIVA

Operación	Costo en este código	Condición
get y set	O(z)	Buscar entre tripletas
Transposición	$O(z^2)$ como cota	Cada set busca duplicados
Producto Ax	O(r + z)	Inicializa r resultados
Suma	$O((zA + zB)^2)$ como cota	Inserciones con búsqueda
Espacio activo conceptual	O(z)	Más metadatos y capacidad

4.6 Resultado Esperado

El estudiante debe recuperar cero en A`[2,3]`, obtener cinco entradas activas, reconstruir la transpuesta correctamente y explicar por qué Ax produce `[36,43,0,57]`. Las cotas se refieren a una versión general con capacidad suficiente; el arreglo de 20 posiciones limita esta demostración.

Verificación	Resultado
Inserción y actualización	Una entrada por coordenada
Eliminación	No guarda el cero
Producto	36, 43, 0, 57
Suma	Cancelación sin duplicados

4.7 Aplicación Práctica

Las matrices dispersas permiten representar conexiones, frecuencias de palabras y relaciones con muchos ceros. Antes de omitir una entrada hay que definir qué significa su ausencia: un dato desconocido no equivale necesariamente a cero. Para cálculos repetidos por filas, CSR puede ser más apropiado que el COO de construcción.

Paso 6 — Resultados y cierre (5 min)

En parejas, una persona explica una tripleta y otra verifica el cálculo de una fila. Intercambiar funciones para que ambas puedan justificar la respuesta. Cerrar preguntando cuándo se elimina una entrada y por qué la transpuesta cambia las dimensiones.

5. Recomendaciones al siguiente tutor

Dificultades previstas:

- Los errores previsibles son mezclar los arreglos paralelos, cambiar filas por columnas y disminuir n al actualizar. En una estructura ortogonal aparece otra dificultad: desconectar un nodo solo de uno de sus recorridos.

Recomendaciones para futuras sesiones:

- Dibujar la matriz densa pequeña antes de mostrar las tripletas.
- Reservar el código para las operaciones centrales y preparar `Main.java` de antemano.

- Pedir que dos estudiantes expliquen una misma fila del producto y revisen sus índices.
- Contrastar las cotas de la implementación sencilla con las de formatos optimizados.

6. Referencias

1. Netlib. Templates for the Solution of Linear Systems. Compressed Row Storage. https://netlib.org/linalg/html_templates/node91.html
2. SciPy. Sparse Arrays. Documentación oficial. https://docs.scipy.org/doc/scipy/tutorial/sparse.html
3. Material de clase. Matriz dispersa, unidad 4. Laboratorio de Estructuras de Datos C, Escuela de Ingeniería en Ciencias y Sistemas, USAC, 2026.